

23 02 26

Discreto, Dinámico, Determinista, Conocido

> Planeación: encontrar un plan que
me lleve de un estado inicial a un
estado terminal con el menor costo

proceso de transición de Markov MDP

Lo que sigue: Discreta, Dinámico, Estocástico,
Conocido

→ Discreto, Dinámico, Estocástico, Desconocido

Caso multiagente: El gato

Cuando tienes más de un agente se le
llama juego

> Zero Sum Games

> General Sum Games

24 feb 2026

class Juego2T7D (Object):

def acciones legales (estado, jugador):
return conjunto, self, acciones

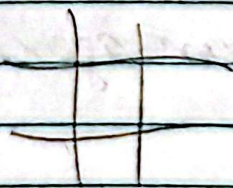
def sucesor (estado, accion, jugador):
self, return estado nuevo

def terminal (estado):
self

return bool

def ganancia (self, estado):

return ganancia humano (ganancia P1)



$S = \{S_0, \dots, S_8\}$

$S_i \in \{1, 0, -1\}$

$a_i \in \{0, \dots, 8\}$

class Gato (Juego2T7D):

def acciones legales (self, estado, jugador):

return [i for i in range (9) if
estado[i] == 0]

o (i for (i, s) in enumerate
(estado) if s == 0)]

def sucesor (self, estado, accion,
jugador):

s_n = estado [:]

s_n[accion] = jugador

return s_n

24 Feb 26

def terminal (self, estado):

Quitar
ganancia
cta

if 0 no + in estado:

return True ← 0 en ganancias

if ((s[0] == s[4] == s[5]) != 0 or

s[2] == s[4] == s[6])) and

s[4] != 0

return True, return s[4] en ganancias

for i in range(3):

if ((s[i] == s[i+3] == s[i+6]) or

((s[3i] == s[3i+1] == s[3i+2])) and

s[i] == 0

return True

return False

def ganancias

24 feb 26

gato = Gato()

humano = 1 # 1 o -1

~~for in range~~

S = [0] * 9

j = 1 → actualiza_tablero(S).

for in range(9):

if j == humano:

acciones = gato.acciones legales(S, j)

a = escoge_accion(acciones)

~~S = gato.S~~

else:

a = agente(gato, S, j)

S = gato.succesor(S, a, j)

actualiza_tablero(S)

~~if gato.terminal~~

j = -j

if gato.terminal(S):

break

~~else~~

actualiza_score(gato.ganancia(S))